

Islamic University of Technology

Lab: Greedy & Flow-Network Engineering

CSE 4810: Algorithm Engineering
Group 1 — Prim's Algorithm & Max-Flow

ASH, ABT(PT)
July 29, 2026

General Instruction

This lab consists of **three tasks** and is designed to fit a **2-hour** session. Written derivations are replaced by **live implementation, profiling, and verbal defense**. For every task you must:

- Step 1.** Complete the `// TODO` sections in the provided C++ source file(s).
- Step 2.** Compile and run the program, feeding it the exact stress-test / case-study inputs specified.
- Step 3.** Produce the requested live artifact — a runtime plot, a max-flow value, or a set of bottleneck edges — and keep your terminal/IDE open.
- Step 4.** Be ready to defend your engineering choices verbally using the artifact on screen; no written report is collected.

Task 1 — Prim's Algorithm	~35 minutes
Task 2 — Max-Flow (Edmonds-Karp)	~55–60 minutes
Task 3 — Live Profiling & Case Study	~25–30 minutes

Task 2 is the heaviest task — budget your time accordingly. If you are running short, prioritize a *correct* Max-Flow implementation over a fully-polished Task 3 plot.

Files Provided

<code>PrimMST.cpp</code>	Prim's Algorithm skeleton (naive & heap variants)
<code>MaxFlow.cpp</code>	Edmonds-Karp Max-Flow skeleton
<code>plotter_group1.py</code>	Plots runtime data from both tasks

Task 1: Structural Engineering of Prim's Algorithm

Objective: Build a Minimum Spanning Tree using Prim's Algorithm in two variants, and understand *why* the "right" data structure choice depends on graph density.

Variant	Strategy	Why It Matters
prim_naive	Maintains a <code>key[]</code> array and linearly scans all V vertices at every step to find the minimum-key vertex not yet in the MST	Correct , $O(V^2)$. No heap overhead — can actually win on <i>dense</i> graphs where $E \approx V^2$.
prim_heap	Uses a binary min-heap (<code>priority_queue</code>) with lazy deletion to select the next vertex, pushing every relaxed edge	$O(E \log V)$. Wins decisively on <i>sparse</i> graphs, but pays a $\log V$ tax and duplicate-push overhead that can make it <i>slower</i> than naive on dense graphs.

0.1 TODO 1: Implement `prim_naive(n, graph)`

[Hint] Maintain `key[]` (best known edge weight into the MST) and `inMST[]`. At each of the n iterations, scan all vertices once to find the unvisited vertex with minimum `key`, add it, then relax its neighbors. Do not use a heap.

0.2 TODO 2: Implement `prim_heap(n, graph)`

[Hint] Push $(weight, vertex)$ pairs onto a `priority_queue` with `greater<>` for min-ordering. Pop the minimum; if that vertex is already in the MST, discard it (lazy deletion) and pop again. Otherwise add it and push all its outgoing edges.

Sanity Check — Expected Output

```

1 --- PRIM'S MST SANITY CHECK (n=5) ---
2 Naive : 16 (Expected 16)
3 Heap  : 16 (Expected 16, should match Naive)
4 -----

```

Compile & Run

```

1 g++ -O2 -std=c++17 -o prim PrimMST.cpp
2 ./prim

```

Task 2: Max-Flow from Scratch (Edmonds-Karp)

Objective: Implement Ford-Fulkerson using BFS-selected augmenting paths (Edmonds-Karp), building your own residual graph representation from scratch.

Key Idea — Residual Graph

Every directed edge $u \rightarrow v$ with capacity C is stored alongside a reverse edge $v \rightarrow u$ with capacity 0. Pushing f units of flow along $u \rightarrow v$ increases that edge's `flow` by f and decreases the reverse edge's `flow` by f — this is what allows the algorithm to "undo" a bad routing decision later without any special-case logic.

0.3 TODO 1: Implement `bfs_augmenting_path(source, sink, parentEdge)`

[Hint] Standard BFS, but an edge is traversable only if $\text{capacity} - \text{flow} > 0$ (positive residual capacity). Record `parentEdge[v] = {u, edge_index}` for every vertex you visit so the path can be reconstructed by walking backwards from `sink`.

0.4 TODO 2: Implement `max_flow(source, sink)`

[Hint] Loop: call `bfs_augmenting_path`. If it fails, stop. Otherwise walk the path from `sink` back to `source` to find the bottleneck (minimum residual capacity), then walk it again to actually push that much flow onto every edge and subtract it from every reverse edge. Accumulate the bottleneck into the total.

0.5 TODO 3: Implement `find_saturated_edges()`

[Hint] Call this only after `max_flow` has fully terminated. An original edge ($\text{capacity} > 0$) is saturated when $\text{flow} == \text{capacity}$. Skip the artificial reverse edges (they always have $\text{capacity} == 0$).

Sanity Check — Expected Output

```
1 --- MAX-FLOW SANITY CHECK (6 vertices) ---
2 Computed Max-Flow : 23 (Expected 23)
3 -----
```

Case Study — Pipeline Capacity Network

Runs to Perform

- Run `case_study_pipeline()` (already wired into `main`) on the provided 8-node "intake → delivery" network.
- Report the maximum deliverable flow from intake (vertex 0) to the delivery station (vertex 7).
- Report every saturated (bottleneck) edge using your `find_saturated_edges()` implementation — these are the pipe segments that would need capacity upgrades to increase total throughput.

Compile & Run

```
1 g++ -O2 -std=c++17 -o maxflow MaxFlow.cpp
2 ./maxflow
```

Task 3: Live Profiling & Combined Case Study

Objective: Quantify the density crossover between Prim's variants, and connect Max-Flow to a real bottleneck-identification scenario.

Runs to Perform

- **Prim's Density Sweep:** `run_benchmark()` in `PrimMST.cpp` times both variants across increasing n at two densities (sparse ≈ 0.02 , dense ≈ 0.3). Run it and inspect `data_prim.txt`.
- **Plot it:** `python3 plotter_group1.py data_prim.txt data_maxflow.txt` produces `prim_runtime.png` and `maxflow_runtime.png`.
- **Bottleneck Report:** Using your Task 2 case-study output, identify which single saturated edge, if upgraded first, would most likely increase total throughput the most (informal reasoning is fine — no need to re-run max-flow with hypothetical capacities unless you have time).

Core Engineering Defense (Verbal Check)

- **The Crossover:** Point to your sparse-vs-dense plots and identify the graph density at which `prim_heap` stops being the faster choice. Explain *why* lazy deletion causes the heap variant's edge count (and thus push count) to blow up on dense graphs.
- **Residual Graph Logic:** Explain, using your own `add_edge` implementation, why the reverse edge with capacity 0 is necessary — what would break if you omitted it entirely?
- **Bottleneck Reasoning:** Justify, using the saturated-edge list from your case study, which upgrade you would recommend first and why.

The instructor may ask you to add one extra edge to the pipeline network (connecting two previously unconnected intermediate nodes) and predict — *before* re-running your code — whether the max-flow value can increase, decrease, or must stay the same. Justify your answer using the min-cut you would expect to change (or not).